МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №2
з дисципліни: «Безпека програм та даних»
з теми: «Програмна реалізація шифрування даних за допомогою несиметричного криптоалгоритму RSA»

Виконали:
ст. гр. ПЗПІ-23-3
Чуваєв А. О. (вар. 16)
ст. гр. ПЗПІ-23-3
Світенко С. М. (вар. 13)

Перевірив:
Старший викладач кафедри ПІ
Олійник О. О.

Харків – 2026

2 ПРОГРАМНА РЕАЛІЗАЦІЯ ШИФРУВАННЯ ДАНИХ ЗА ДОПОМОГОЮ НЕСИМЕТРИЧНОГО КРИПТОАЛГОРИТМУ RSA

2.1 Мета роботи

Дослідити принципи функціонування несиметричних криптографічних систем з відкритим ключем, розробити клієнт-серверний програмний комплекс для захищеного двостороннього обміну конфіденційними даними через незахищений канал зв'язку на базі алгоритму RSA, вирішити проблему міжплатформної стандартизації при взаємодії систем, розроблених на двох різних мовах програмування (.NET 10 / C# та JavaScript / Web Crypto API) з використанням незалежних криптографічних бібліотек та розміру ключів 2048 бітів.

2.2 Індивідуальне завдання та технічні вимоги

Робота виконується бригадою з двох студентів групи ПЗПІ-23-3:

- **Студент 1:** Чуваєв Артем Олександрович (Варіант 16).
- **Студент 2:** Світенко Софія Миколаївна (Варіант 13).

Технічні вимоги до програмної реалізації:

а) **Різні мови програмування:** дві сторони мережевого обміну повинні бути розроблені на принципово різних мовах програмування:
   - Серверна сторона: **C# (.NET 10)**.
   - Клієнтська сторона: **JavaScript (ES2022 / Web Crypto API)**.

б) **Різні криптографічні бібліотеки та проблема стандартизації:**
   - Сервер використовує стандартний системний модуль «.NET» – «System.Security.Cryptography.RSA».
   - Клієнт використовує стандартизований браузерний API – «window.crypto.subtle» (W3C Web Crypto API).
   - Основне завдання – дослідити та розв'язати проблему стандартизації форматів ключів (SPKI / PKCS#8 PEM), схем набивання (RSA-OAEP-SHA256) та обмеження довжини повідомлення.

в) **Розмір ключів:** розмір ключів обов'язково має перевищувати 1024 біти. У роботі використано криптостійкий промисловий стандарт **2048 бітів** (з експонентою $E = 65537$).

г) **Повний двосторонній зашифрований обмін:** зашифрований трафік передається в обидва боки:
   - Клієнт шифрує логін, пароль та нотатку відкритим ключем сервера ($\text{Server}_{\text{pub}}$). Сервер розшифровує їх своїм закритим ключем ($\text{Server}_{\text{priv}}$).
   - Сервер шифрує сформовану відповідь (статус, токен авторизації, часову мітку) відкритим ключем клієнта ($\text{Client}_{\text{pub}}$). Клієнт розшифровує її своїм закритим ключем ($\text{Client}_{\text{priv}}$).

д) **Інтерфейс користувача:** розробка зручного, інтуїтивного Web-інтерфейсу з відображенням ключів у форматі PEM, форми вводу, візуального перехоплення трафіку в незахищеному каналі та журналу криптографічних подій у реальному часі.

2.3 Теоретичні відомості

2.3.1 Концепція асиметричної криптографії

На відміну від симетричних алгоритмів (DES, AES), де один і той самий секретний ключ використовується і для зашифрування, і для розшифрування, асиметричні шифри використовують узгоджену пару математично пов’язаних ключів:

- **Відкритий ключ (Public Key):** є загальнодоступним, публікується або передається незахищеними каналами зв’язку. Використовується для шифрування даних або верифікації цифрового підпису.
- **Закритий / секретний ключ (Private Key):** зберігається власником у суворій таємниці. Використовується для розшифрування шифротекстів або накладання цифрового підпису.

Головна перевага асиметричної криптографії полягає у відсутності потреби в попередній передачі спільного таємного ключа захищеним каналом (вирішення фундаментальної проблеми розподілу ключів).

2.3.2 Математичні засади криптосистеми RSA

В основі криптографічного алгоритму RSA (Рон Рівест, Аді Шамір, Леонард Адлеман, 1977 р.) лежить обчислювальна складність задачі факторизації великих складених чисел – розкладання добутку двох надвеликих простих чисел на множники.

1. Генерація ключової пари:

а) Генеруються два випадкових незалежних великих простих числа P та Q однакової бітової довжини (по 1024 біти для отримання 2048-бітного ключа).

б) Обчислюється модуль системи N:

$$N = P \cdot Q. \tag{2.1}$$

Довжина модуля N у бітах є розрядністю ключа (2048 бітів = 256 байтів).

в) Обчислюється значення функції Ейлера $\varphi(N)$ (кількість натуральних чисел, менших за N і взаємно простих з ним):

$$\varphi(N) = (P-1)(Q-1). \tag{2.2}$$

г) Обирається відкрита експонента E, яка задовольняє умови $1 < E < \varphi(N)$ та НСД$(E, \varphi(N)) = 1$. За загальновизнаним криптографічним стандартом як E обирається просте число Ферма $F_4 = 2^{16} + 1 = 65537$ («0x10001»), яке містить лише дві двійкові одиниці, що істотно прискорює операцію модульного піднесення до степеня за алгоритмом «Square-and-Multiply».

д) За допомогою розширеного алгоритму Евкліда обчислюється секретна експонента D, що є мультиплікативно оберненою до E за модулем $\varphi(N)$:

$$D \cdot E \equiv 1 (\mathrm{mod}\, \varphi(N)) \quad \Rightarrow \quad D = E^{-1} \bmod \varphi(N). \tag{2.3}$$

Ключові структури:

– **Відкритий ключ:** пара (E, N).

– **Закритий ключ:** пара (D, N) (разом із компонентами $P, Q, d_P, d_Q, q_{\text{inv}}$).

2. Шифрування та розшифрування: Відкрите повідомлення розглядається як ціле число M ($0 \le M < N$).

– **Шифрування (публічним ключем):**

$$C \equiv M^E (\mathrm{mod}\, N). \tag{2.4}$$

– **Розшифрування (приватним ключем):**

$$M \equiv C^D (\mathrm{mod}\, N). \tag{2.5}$$

Коректність відновлення відкритого тексту базується на теоремі Ейлера ($M^{\varphi(N)} \equiv 1 (\mathrm{mod}\, N)$ для НСД$(M, N) = 1$):

$$C^D \equiv \left(M^E\right)^D \equiv M^{ED} \equiv M^{1+k\varphi(N)} \equiv M \cdot \left(M^{\varphi(N)}\right)^k \equiv M \cdot 1^k \equiv M (\mathrm{mod}\, N). \tag{2.6}$$

2.3.3 Проблема стандартизації та інтероперабельності між платформами

При практичній інтеграції систем на різних мовах програмування (.NET та JS) виникають три критичні бар'єри сумісності:

1. Стандарти кодування та форматів ключів:

– Історичний формат **PKCS#1** (RFC 8017) описує виключно структуру RSA-ключів («-----BEGIN RSA PUBLIC KEY-----»).

– Сучасний кросплатформний стандарт **X.509 SubjectPublicKeyInfo (SPKI)** (RFC 5280) є універсальним контейнером для довільних публічних ключів із зазначенням ідентифікатора алгоритму (OID «1.2.840.113549.1.1.1» для «rsaEncryption») у нотації ASN.1 («-----BEGIN PUBLIC KEY-----»).

– У роботі реалізовано узгоджений експорт/імпорт саме у стандарті SPKI PEM у форматі Base64, що гарантує безшовне розпізнавання відкритих ключів як методом «RSA.ImportFromPem()» у .NET, так і «crypto.subtle.importKey("spki", ...)» у Web Crypto API.

2. Схема набивання (Padding) та захист від атак: «Підручниковий» (текстовий) RSA ($C = M^E \bmod N$) без набивання є детермінованим і вразливим до атак на основі обраного шифротексту, гомоморфних атак та атак за малим показником. Застарілий стандарт PKCS#1 v1.5 вразливий до адаптивних атак Блейхенбахера (Bleichenbacher attack).

Тому в нашій реалізації застосовано найбільш криптостійку схему набивання – **RSA-OAEP (Optimal Asymmetric Encryption Padding)** за стандартом PKCS#1 v2.2 / RFC 8017 з геш-функцією **SHA-256**. Схема OAEP додає маскування на основі криптографічного генератора випадкових чисел (MGF1), перетворюючи

RSA на ймовірнісну криптосистему зі строгою індиференційованістю за адаптивно обраним шифротекстом (IND-CCA2).

3. Обмеження довжини відкритого тексту та блокове розбиття (Chunking): Математично вхідне число M має бути строго меншим за модуль N. При використанні схеми OAEP-SHA256 частина розрядів 256-байтного блоку відводиться під геш-мітки та випадкову маску. Максимальна довжина корисного відкритого тексту в одному 2048-бітному блоці становить:

$$L_{\max} = \text{KeySizeBytes} - 2 \cdot \text{HashLen} - 2 = 256 - 2 \cdot 32 - 2 = 190 \text{ байтів}. \quad (2.7)$$

Якщо спробувати зашифрувати повідомлення довжиною понад 190 байтів одним викликом, криптографічні підсистеми Windows CNG та Web Crypto викидають виключення («Invalid parameter 0xc100000d» / «DataError»).

Для подолання цього обмеження в обох компонентах системи (.NET і JS) реалізовано повноцінний алгоритм блокового розбиття:

- Відкритий текст розбивається на сегменти розміром до 190 байтів.
- Кожен сегмент індивідуально шифрується алгоритмом RSA-OAEP-SHA256, генеруючи рівно 256 байтів шифротексту.
- При дешифруванні вхідний масив розбивається на фіксовані блоки по 256 байтів, кожен з яких окремо розшифровується і склеюється у вихідний рядок UTF-8.

2.4 Опис архітектури розробленого програмного забезпечення

Архітектура комплексу базується на децентралізованій клієнт-серверній моделі, де обидва вузли є криптографічно рівноправними та виконують пряме й зворотне асиметричне шифрування.

Порівняльна характеристика компонентів системи наведена в таблиці нижче:

Параметр	**Клієнтська сторона**	**Серверна сторона**
Мова програмування	JavaScript (ES2022, браузер)	C# (.NET 10, автономний додаток)
Криптографічна бібліотека	W3C Web Crypto API («SubtleCrypto»)	«System.Security.Cryptography» (.NET)
Ключова пара RSA	RSA-2048 ($\text{Client}_{\text{pub}}$, $\text{Client}_{\text{priv}}$, $E = 65537$)	RSA-2048 ($\text{Server}_{\text{pub}}$, $\text{Server}_{\text{priv}}$, $E = 65537$)

Параметр	Клієнтська сторона	Серверна сторона
Формат відкритого ключа	X.509 SubjectPublicKeyInfo (SPKI PEM)	X.509 SubjectPublicKeyInfo (SPKI PEM)
Схема набивання	RSA-OAEP з геш-функцією SHA-256	RSA-OAEP з геш-функцією SHA-256
Виконувані функції	Генерація пари, блокове шифрування запитів, дешифрування відповідей	Обслуговування HTTP API, дешифрування запитів, шифрування відповідей, хостинг UI

Специфікація протоколу двостороннього обміну через незахищений канал:

Крок	Дія та напрямок передачі в мережі	Криптографічний захист
1	Клієнт ⟶ Сервер: «GET /api/ handshake»	Ініціалізація зв’язку (відкритий запит)
2	Клієнт ⟵ Сервер: передача відкритого ключа	Відкритий ключ сервера ($Server_{pub}$, SPKI PEM)
3	Клієнт ⟶ Сервер: «POST /api/secure-exchange»	Шифротекст: RSA-OAEP-SHA256 (на $Server_{pub}$)
4	Клієнт ⟵ Сервер: зашифрована відповідь	Шифротекст: RSA-OAEP-SHA256 (на $Client_{pub}$)

а) **Серверна частина («lab2/src/server/»):**

– «RsaCryptoService.cs»: інкапсулює роботу з об’єктом «RSA.Create(2048)». Реалізує методи експорту відкритого ключа у форматі SPKI PEM («ExportSubjectPublicKeyInfoPem»), блокового розшифрування вхідних пакетів «DecryptFromClient» та блокового шифрування відповідей відкритим ключем клієнта «EncryptForClient» з набиванням «RSAEncryptionPadding.OaepSHA256».

– «Program.cs»: високоефективний автономний HTTP-сервер на базі «HttpListener». Роздає статичні клієнтські файли (HTML, CSS, JS), забезпечує обробку CORS-заголовків, обслуговує

ендпоінти «/api/handshake» та «/api/secure-exchange», а також містить автономний режим самотестування («--demo»).

б) **Клієнтська частина («lab2/src/client/»):**

– «index.html»: адаптивна панель приладів з індикатором стану сервера, переглядачами ключів, формою автентифікації, швидкими пресетами бригади та покроковим криптографічним монітором трафіку.

– «style.css»: кібербезпековий інтерфейс із темною темою, неоновими акцентами та моноширинними блоками коду.

– «client.js»: реалізація генерації ключів «window.crypto.subtle.generateKey», імпорту/експорту PEM через «ArrayBuffer», блокового шифрування «encryptChunks» та розшифрування «decryptChunks».

2.5 Результати виконання та протокол роботи програми

2.5.1 Демонстрація для Чуваєва А. О. (Варіант 16)

– **Логін користувача:** «chuvaiev_artem»

– **Пароль:** «Art16_Pass_@2026»

– **Конфіденційні дані:** «Чуваєв Артем (вар. 16): Асиметрична автентифікація RSA-2048 між C# та JS»

1. Вихідний відкритий запит клієнта (Plaintext JSON):

```
{
  "login": "chuvaiev_artem",
  "password": "Art16_Pass_@2026",
  "note": "Чуваєв Артем: перевірка повної двосторонньої
інтероперабельності RSA між .NET 10 та JS",
  "clientTimestamp": "2026-10-29T10:37:47.000Z",
  "protocol": "RSA-OAEP-SHA256"
}
```

2. Пакет у незахищеному мережевому каналі (Клієнт → Сервер, Base64 Ciphertext):

```
W7g6kMaf4/F27wWibrkWW9AzOk+5vVPBl7Gn/3LkuVwrlYAen2... (розмір 684
символи Base64 / 2 блоки по 256 байтів)
```

Примітка: Будь-який сторонній спостерігач або зловмисник у мережі бачить виключно псевдовипадковий бінарний масив і не здатний прочитати логін, пароль чи примітку без приватного ключа сервера.

3. Зашифрована відповідь сервера у мережі (Сервер → Клієнт, Base64 Ciphertext):

```
Pz0mSHekeyCAEgzwH4FGBjhGITDx6RE+bu9Ze6dbXT+7yRy2ce... (розмір 684
символи Base64 / 2 блоки по 256 байтів)
```

4. Розшифрована клієнтом відповідь сервера (Plaintext JSON):

```
{
  "status": "AUTH_OK",
  "user": "chuvaiev_artem",
  "token": "2f6a3103827d",
  "time": "10:37:47",
  "note": "Чуваєв Артем: перевірка пов...",
  "msg": "Двосторонній обмін RSA-2048 підтверджено"
}
```

Результат: Повна автентичність даних, 100% успішне двостороннє криптографічне перетворення.

2.5.2 Демонстрація для Світенко С. М. (Варіант 13)

- **Логін користувача:** «svitenko_sofiia»
- **Пароль:** «Sofi13_Pass_@2026»
- **Конфіденційні дані:** «Світенко Софія (вар. 13): перевірка несиметричного шифрування RSA-2048»

Розшифрована клієнтом відповідь сервера (.NET → JS):

```
{
  "status": "AUTH_OK",
  "user": "svitenko_sofiia",
  "token": "686344f82cb8",
  "time": "10:37:57",
  "note": "Світенко Софія (вар. 13): п...",
  "msg": "Двосторонній обмін RSA-2048 підтверджено"
}
```

Результат: Успішне підтвердження автентифікації та двостороннього шифрування для варіанту 13.

2.6 Відповіді на контрольні запитання

1. Які криптографічні шифри називають несиметричними? Несиметричними (або асиметричними) називають криптографічні алгоритми, в яких для шифрування та розшифрування даних використовуються два різних, але математично взаємопов'язаних ключі: відкритий (public key) та закритий (private key). Знання відкритого ключа дає можливість будь-кому зашифрувати повідомлення, проте розшифрувати його може лише володар закритого ключа. Обчислення закритого ключа з відкритого є математично нездійсненним за розумний час при сучасному рівні обчислювальної техніки.

2. На якій складній задачі засновано алгоритм RSA? Алгоритм RSA засновано на обчислювальній складності **задачі факторизації великих цілих чисел** (розкладання складеного числа на прості співмножники). Помножити два великих простих числа P та Q і знайти їхній добуток $N = P \cdot Q$ обчислювально просто і швидко ($O(n^2)$), проте знайти самі множники P та Q, знаючи лише модуль N довжиною 2048 бітів, для сучасних комп'ютерів є практично неможливим завданням (найкращий відомий алгоритм факторизації – загальний метод решета числового поля GNFS має субекспоненційну складність).

3. Як генеруються відкритий та секретний ключі?

а) Випадковим чином обираються два великих простих числа P та Q.

б) Обчислюється модуль $N = P \cdot Q$ та значення функції Ейлера $\varphi(N) = (P-1)(Q-1)$.

в) Обирається ціле число E таке, що $1 < E < \varphi(N)$ та НСД$(E, \varphi(N)) = 1$ (стандартно $E = 65537$).

г) За розширеним алгоритмом Евкліда знаходиться секретна експонента D, що задовольняє умові $D \cdot E \equiv 1 (\mathrm{mod}\, \varphi(N))$.

д) Пара (E, N) формує відкритий ключ (Public Key), а пара (D, N) – секретний ключ (Private Key). Початкові числа P, Q та значення $\varphi(N)$ обов'язково знищуються або тримаються в суворій таємниці.

4. Як проходить шифрування та розшифрування по алгоритму RSA?

– **Шифрування:** відкрите повідомлення попередньо доповнюється випадковою маскою за схемою OAEP і перетворюється на ціле

число M ($0 \leq M < N$). Шифротекст обчислюється шляхом модульного піднесення до степеня з використанням відкритого ключа:

$$C = M^E \bmod N. \tag{2.8}$$

– **Розшифрування:** володар секретного ключа D обчислює модульне піднесення шифротексту C до степеня D:

$$M = C^D \bmod N, \tag{2.9}$$

після чого знімається набивання OAEP і відновлюється вихідний відкритий текст.

5. Які є недоліки у алгоритмів з відкритим ключем?

а) **Низька обчислювальна швидкодія:** асиметричні операції з великими числами виконуються у 100–1000 разів повільніше за симетричні блокові шифри (AES).

б) **Обмеження розміру даних:** алгоритм не здатний зашифрувати за один прохід дані, більші за довжину модуля (для RSA-2048 з OAEP – не більше 190 байтів). Через це у промислових протоколах (TLS/HTTPS) RSA застосовують як гібридну криптосистему – виключно для захищеного обміну симетричними сесійними ключами (Key Encapsulation).

в) **Вразливість до квантових комп'ютерів:** за наявності повноцінного квантового обчислювача алгоритм Шора дозволяє розкласти модуль N на множники за поліноміальний час, що повністю зламує RSA (вимагає переходу на постквантову криптографію PQC).

г) **Необхідність використання великих ключів:** для еквівалентної стійкості до 128-бітного AES необхідний RSA-ключ розміром не менше 3072 бітів.

6. Що мається на увазі під поняттям незворотної функції? Під незворотною (або односпрямованою) функцією (One-Way Function, OWF) розуміють математичну функцію $y = f(x)$, яку легко обчислити для будь-якого вхідного аргументу x за поліноміальний час, але для якої за відомим значенням y обчислювально неможливо за прийнятний час знайти початковий прообраз x (тобто знайти обернену функцію $f^{-1}(y)$). У криптографії RSA використовується різновид односпрямованих функцій – **односпрямована функція з потайним ходом (Trapdoor One-Way Function)**, обернення якої є неможливим для всіх, крім особи,

яка володіє спеціальним секретом («лазівкою» або trapdoor, яким у RSA виступає секретна експонента D, обчислена на основі факторизації модуля N).

2.7 Висновки

У ході виконання лабораторної роботи:

а) Досліджено теоретичні та практичні засади функціонування асиметричного криптоалгоритму RSA, зокрема використання односпрямованих функцій з потайним ходом на основі задачі факторизації чисел.

б) Розроблено повноцінний кросплатформний клієнт-серверний комплекс на базі двох принципово різних мов програмування: серверної підсистеми на C# (.NET 10) та клієнтської частини на JavaScript (ES2022).

в) Успішно розв’язано проблему міжплатформної стандартизації при взаємодії незалежних криптографічних бібліотек («System.Security.Cryptography» та W3C «Web Crypto API»): узгоджено формати відкритого ключа SPKI PEM (X.509), сучасну схему набивання RSA-OAEP з геш-функцією SHA-256 та схему модульного розбиття пакетів на сегменти по 190 байтів.

г) Забезпечено повний двосторонній зашифрований обмін через незахищений канал зв’язку (як зашифрування облікових даних клієнтом на публічному ключі сервера, так і зашифрування відповіді сервером на публічному ключі клієнта).

д) Створено естетичний та інформативний Web-інтерфейс із засобами перевірки зв’язку, швидкими пресетами для бригади, інспекцією ключів, наочним моніторингом перехоплення шифротекстів у каналі зв’язку та журналом аудиту подій.

2.8 Додаток А. Посилання на репозиторій з вихідним кодом

Повний вихідний код розробленого клієнт-серверного додатку доступний у віддаленому репозиторії GitHub:

– **Каталог вихідного коду лабораторної роботи №2:**
https://github.com/tmprokill/bptd_4_1/tree/main/lab2/src

Прямі посилання на компоненти системи:

а) **RsaCryptoService.cs** – серверний криптографічний сервіс RSA на .NET: генерація пари 2048 бітів, експорт SPKI PEM, блокове шифрування та розшифрування за схемою RSA-OAEP-SHA256.

б) **Program.cs** – серверний додаток .NET 10 («HttpListener»): обслуговування REST API ендпоінтів «/api/handshake» та «/api/secure-exchange», хостинг статичного веб-інтерфейсу, консольний журнал та режим самотестування («--demo»).

в) **client.js** – клієнтський JavaScript модуль на базі W3C Web Crypto API: генерація ключів RSA-2048, конвертація SPKI PEM, блокове шифрування/розшифрування запитів і відповідей, інтерактивне керування інтерфейсом.

г) **index.html** – розмітка веб-інтерфейсу з панелями ключів, формою автентифікації, перехоплювачем відкритого каналу зв’язку та журналом аудиту.

д) **style.css** – стилізація сучасного кібербезпекового вебінтерфейсу користувача.